

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	CH.Bhavya Sri	Reg. No.:	192411249
Section / Batch:	C	Date:	08.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Question:

Analyze how Binary Search improves search efficiency. Identify requirements such as sorted indexing and accuracy. Apply divide and conquer concepts. Analyze time complexity and justify the solution.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				

Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

The screenshot shows the SIMATS Online Programming Lab interface. The header includes the SIMATS Engineering logo, the lab name "SIMATS Online Programming Lab", and the user "CHENNABOINA BHAVYA SRI". The main section is titled "CO3 AT2-EXPERIMENTS" and shows a question "Q1 University Course Lookup System (Binary Search)". The question text is: "A university system maintains sorted course records and requires fast retrieval. 100 marks". The input field contains "5", and the output field shows "Course found at index 2". The code editor shows a Python function for binary search.

Q1 University Course Lookup System (Binary Search) A university system maintains sorted course records and requires fast retrieval. 100 marks

Analyze how Binary Search improves search efficiency. Identify requirements such as sorted indexing and accuracy. Apply divide and conquer concepts. Analyze time complexity and justify the solution.

Your Code

```
1 def binary_search_course(course_list, target_course):
2     low = 0
3     high = len(course_list) - 1
4
5     while low <= high:
6         mid = (low + high) // 2
7
8         if course_list[mid] == target_course:
9             return mid
10        elif course_list[mid] < target_course:
11            low = mid + 1
12        else:
13            high = mid - 1
```

Input

5
Algorithms
Database
Networks

Output

Course found at index 2

Problem Statement

A university maintains a **sorted list of course records**. To quickly find whether a particular course exists in the list, **Binary Search** is used.

Binary Search follows the **Divide and Conquer** approach. Instead of checking every course one by

one, it repeatedly divides the search range into two halves.

1. Objective

To implement a **Binary Search algorithm** for searching a course in a sorted list and analyze its efficiency.

Requirements

- Course records must be **sorted**.
 - The search should be accurate.
 - Binary Search should be used instead of Linear Search.
 - The algorithm should follow the **Divide and Conquer** technique.
 - Analyze the time complexity.
-

2. How Binary Search Works

Suppose the sorted course list is:

Algorithms

Database

Networks

Operating Systems

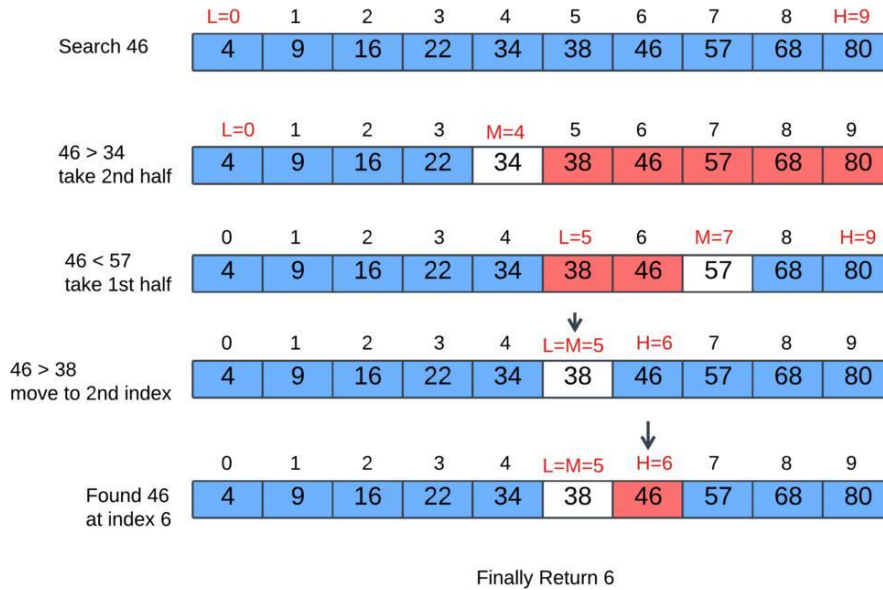
Python

If we want to search for **Networks**:

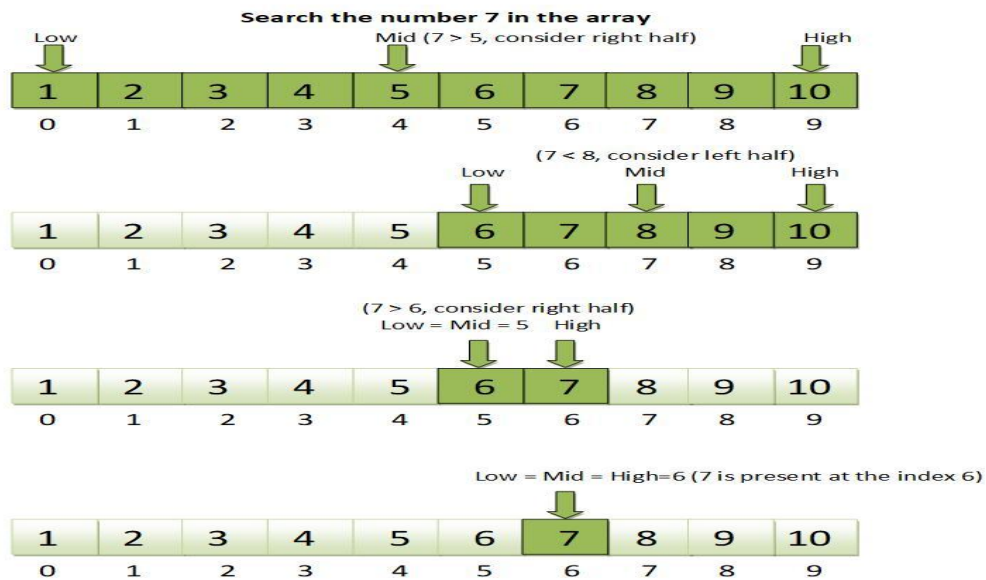
1. Find the middle element.
2. Compare it with the target.
3. If the target is equal → course found.
4. If the target is greater → search the right half.
5. If the target is smaller → search the left half.

6. Repeat until the course is found or the search range becomes empty.

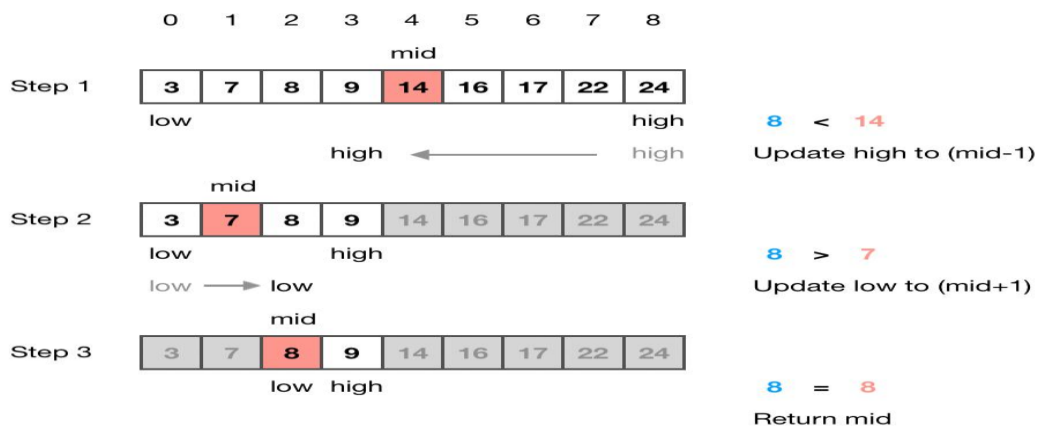
Binary Search



Binary Search



Find 8



3. Algorithm

Step 1: Start

Step 2: Set low = 0

Step 3: Set high = length of course list - 1

Step 4: Repeat while low <= high

Step 5: Find middle = (low + high) // 2

Step 6: Compare middle element with target course

Step 7: If equal, return the index

Step 8: If middle element < target, search right half

Step 9: Otherwise, search left half

Step 10: If low > high, course is not found

Step 11: Stop

4. Python Program

```
def binary_search_course(course_list, target_course):
```

```
    low = 0
```

```
    high = len(course_list) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if course_list[mid] == target_course:
```

```
            return mid
```

```
        elif course_list[mid] < target_course:
```

```
            low = mid + 1
```

```
    else:
```

```
        high = mid - 1
```

```
return -1
```

```
# Input
```

```
course_list = ["Algorithms", "Database", "Networks",  
              "Operating Systems", "Python"]
```

```
target_course = "Networks"
```

```
# Search
```

```
result = binary_search_course(course_list, target_course)
```

```
if result != -1:
```

```
    print("Course found at index", result)
```

```
else:
```

```
    print("Course not found")
```

Output

Course found at index 2

5. Divide and Conquer Concept

Binary Search uses **Divide and Conquer**:

Divide

The list is divided into two halves.

Conquer

Only the half that may contain the target is searched.

Repeat

The selected half is again divided into two halves until the target is found.

For example:

5 courses

↓

2 halves

↓
Search one half
↓
Divide again
↓
Find target

6. Time Complexity

Case	Time Complexity
------	-----------------

Best Case	$O(1)$
-----------	--------------------------

Average Case	$O(\log n)$
--------------	-------------------------------

Worst Case	$O(\log n)$
------------	-------------------------------

Space Complexity **$O(1)$**

Why **$O(\log n)$** ?

For every comparison, Binary Search eliminates approximately **half of the remaining elements**.

For example:

1000 elements

↓
500
↓
250
↓
125
↓
62
↓
31
↓
...

Therefore, the number of comparisons grows logarithmically:

Time Complexity = $O(\log n)$

7. Advantages

- Very fast for large sorted datasets.
- Requires fewer comparisons than Linear Search.
- Efficient course lookup.
- Uses the Divide and Conquer strategy.
- Space-efficient when implemented iteratively.

8. Limitation

The main requirement is that the **course list must already be sorted**. If the records are unsorted, Binary Search cannot be directly applied.

9. Conclusion

Binary Search is an efficient searching technique for sorted university course records. It repeatedly divides the search space into two halves, making it much faster than Linear Search for large datasets. Its worst-case time complexity is **$O(\log n)$** , making it suitable for fast course retrieval systems.

Binary Search is an efficient searching technique used to quickly locate a course in a sorted university course list. It follows the Divide and Conquer approach by repeatedly dividing the search space into two halves and eliminating the unnecessary half after each comparison. Compared with Linear Search, it significantly reduces the number of comparisons, especially for large datasets. The algorithm has a time complexity of **$O(\log n)$** in the average and worst cases and **$O(1)$** space complexity when implemented iteratively. Therefore, Binary Search provides a fast, accurate, memory-efficient, and scalable solution for university course lookup systems and other applications involving sorted data.